Ghost Gutzwiller Approximation for the Half-Filled Hubbard Dimer: Equations, Conventions, and Numerical Implementation

gGut_mau project — `gga_dimer.py`

October 19, 2026

Contents

1 Physical model **2**

2 The ghost Gutzwiller ansatz: general framework **2**

3 Theory: the constrained variational Lagrangian **3**
3.1 Euler–Lagrange equations 5

4 General workflow: inputs, guesses, and order of computation **6**

5 Specialization to the two-site, two-independent-fragment dimer **9**
5.1 Each site is its own fragment; the two fragments need not be equal 9
5.2 Quasiparticle Hamiltonian for two independent fragments 9
5.3 Quasiparticle ground state and occupation rule 9

6 Self-consistency equations **10**
6.1 Hybridization V (Eq. 7 of the reference) 10
6.2 Bath potential λ^c (Eq. 8 of the reference) 11
6.3 Impurity (embedding) Hamiltonian 12
6.4 Update of R and Δ (Eq. 10 of the reference) 15
6.5 Variational energy 16

7 Numerical solution of the self-consistency loop **16**
7.1 Where λ^I comes from: no fit, just Eq. (20) 16
7.2 Outer loop: mixing and convergence 17
7.3 The main self-consistency loop, in pseudocode 17
7.4 Validation against the reference C++ code, iteration by iteration 18

8 Observables reported by the code **20**

9 Post-processing: quasiparticle Green's function and spectral function **20**

10 Command-line interface and input file formats **21**

11 Worked examples and current validation status **22**

12 Are the converged points solutions? Stationarity and second-order tests **23**

13 Generalization: a site-splitting potential ΔV **25**

14 References **26**

Purpose of this note. This document is a pedantic, self-contained description of *exactly* what the code in `gga_dimer.py` computes: every equation, every index convention, every regularization constant, and every numerical safeguard, together with the reasoning behind each choice. It is meant to let a reader reproduce the code from the equations alone, or verify the code against the equations line by line. Physical background (the ghost Gutzwiller approximation, gGA) is kept to the minimum needed to fix notation; references to the original literature are given in Section 14.

1 Physical model

We study the two-site ("dimer") single-band Hubbard Hamiltonian

$$H_{\text{phys}} \;=\; -t \sum_{\sigma=\uparrow,\downarrow} \left(c^{\dagger}_{0\sigma} c_{1\sigma} + c^{\dagger}_{1\sigma} c_{0\sigma} \right) \;+\; U \sum_{i=0,1} n_{i\uparrow} n_{i\downarrow} \;-\; \mu \sum_{i=0,1} \left(n_{i\uparrow} + n_{i\downarrow} \right), \tag{1}$$

with sites labeled $i = 0, 1$, one physical orbital per site ($N_{\text{phys}} = 1$), and both spins present. The hopping amplitude t sets the energy unit; the code always uses $t = 1$ unless stated otherwise (the CLI flag `--t` allows changing it, and every energy reported by the code is implicitly in units of t).

Half filling and particle-hole symmetry. The chemical potential is fixed to the particle-hole symmetric value

$$\mu = U/2, \tag{2}$$

which pins the total occupation to $\langle n_{0\sigma} + n_{0\bar{\sigma}} + n_{1\sigma} + n_{1\bar{\sigma}} \rangle = 2$ (one electron per spin, summed over both sites) for *any* U. This is the only filling considered by the code; there is no doping/chemical-potential scan.

Exact benchmark. `exact_dimer.py` diagonalizes Eq. (1) exactly in the full $2^4 = 16$-dimensional Fock space of the four physical modes ($i \in \{0, 1\}$, $\sigma \in \{\uparrow, \downarrow\}$), by building the four fermionic annihilation operators via an explicit Jordan–Wigner string (see Section 6.3 for the general construction, which is shared with the impurity solver). It reports the ground-state energy E_0, total filling $\langle n \rangle$, and the double occupancy per site $\text{docc} = \frac{1}{2} \sum_i \langle n_{i\uparrow} n_{i\downarrow} \rangle$, used throughout as the ground truth that the gGA approximation is checked against.

2 The ghost Gutzwiller ansatz: general framework

The ghost Gutzwiller approximation (gGA) approximates the ground state of Eq. (1) variationally as $|\Psi_G\rangle = P|\Psi_{\text{qp}}\rangle$, where $|\Psi_{\text{qp}}\rangle$ is the Slater-determinant ground state of an auxiliary, purely quadratic *quasiparticle* Hamiltonian H_{qp}, and P is a Gutzwiller-type projector onto the physical Hilbert space. In the standard ("rotationally invariant slave boson"/gGA) formulation, the projector is replaced by an equivalent *embedding* (impurity) problem, and the whole scheme reduces to a self-consistency loop between:

1. a one-body quasiparticle Hamiltonian $H_{\rm qp}(R, \lambda)$, built from a *renormalization matrix* R and an *onsite potential* λ, and

2. a many-body *impurity* Hamiltonian $H_{\rm imp}(V, \lambda^c)$, describing one physical (correlated) orbital coupled to $N_{\rm eff}$ auxiliary ("ghost") bath orbitals through a hybridization V and a bath-onsite potential λ^c.

Dimension bookkeeping. Let $N_{\rm phys}$ be the number of physical orbitals per fragment (here $N_{\rm phys} = 1$: one site, one orbital, spin handled symmetrically) and let N_g be the number of *extra* ghost bath orbitals requested by the user, on top of the single bath orbital that even the *non-ghost* (ordinary) Gutzwiller/slave-boson theory already has. Define

$$N_{\rm eff} \;=\; N_{\rm phys} + N_g \;=\; 1 + N_g. \tag{3}$$

$N_g = 0$ is *not* "no bath"; it is the ordinary single-bath Gutzwiller theory. Each fragment's embedding Hamiltonian then has $N_{\rm phys} + N_{\rm eff} = 2 + N_g$ orbitals per spin (one physical, $N_{\rm eff}$ bath), so, including both spins, the impurity Fock space has dimension

$$\dim(\mathcal{F}_{\rm imp}) \;=\; 2^{\,2(1+N_{\rm eff})} \;=\; 4^{\,1+N_{\rm eff}} \;=\; 4^{\,2+N_g}. \tag{4}$$

(An earlier draft of the project notes stated 4^{1+N_g}; that is off by one power of 4 and has been corrected in this document and cross-checked against the code, which builds exactly $2(1 + N_{\rm eff})$ fermionic modes – see Section 6.3.)

3 Theory: the constrained variational Lagrangian

The self-consistency equations used throughout this document (Eqs. (15), (20), (36) in Section 6) are not postulated independently: they are the stationarity (Euler–Lagrange) conditions of a single constrained variational functional – a Lagrangian – whose extremization is equivalent to minimizing $\langle\Psi_G|H_{\rm phys}|\Psi_G\rangle$ over the Gutzwiller-projected variational family of Section 2. **Five**, not four, independent Lagrange multipliers appear in this Lagrangian – it is essential to keep all five, because two of them ($E_{\rm qp}$ and $E^{I,c}$) are what *turn out*, only after the Euler–Lagrange equations are taken, to equal the quasiparticle and embedding ground-state energies; they are not postulated as energies from the start. This section reproduces the exact Lagrangian derived in `NotesOngGut.pdf` (its Eq. 29, itself the finite-embedding rewrite of the periodic-lattice Lagrangian, Eq. 19, once the projector matrix ϕ is traded for an explicit embedding wavefunction $|\Phi^I\rangle$ – see also Mejuto-Zaera, arXiv:2403.05157, Eqs. 5–6), specialized here to our two real-space fragments (no Brillouin-zone sum, and the two fragments *not* assumed equal, per Section 5). The precise algebraic form of the resulting equations (fixed by the exact operator-ordering convention already used in $H_{\rm imp}$, Eq. (25), and cross-checked term by term against the independent reference implementation) is worked out operationally in Section 6.

Variational objects. For each fragment $I \in \{0, 1\}$:

- $|\Psi_{\rm qp}\rangle$: the many-body quasiparticle wavefunction – a genuinely independent variational object, *not* yet assumed to be any particular eigenstate of any particular Hamiltonian. (It is only the Euler–Lagrange equation below that forces it to be an eigenstate of $H_{\rm qp}$, and only the physical half-filling requirement, Section 5.3, that then picks out the filled Fermi sea, Eq. (13), as the relevant one.)

- $\Delta^{II} \in \mathbb{R}^{N_{\rm eff}\times N_{\rm eff}}$: the *shared* target one-body density matrix of the $N_{\rm eff}$ ghost/bath orbitals – the single object that both $|\Psi_{\rm qp}\rangle$ and $|\Phi^I\rangle$ below are separately constrained to reproduce (in the code, the fragment-diagonal blocks of the mixed Δ that the outer loop of Section 7.2 carries from one iteration to the next);
- $R^I \in \mathbb{R}^{N_{\rm eff}}$: the renormalization vector entering $H_{\rm qp}$, Eq. (11);
- $|\Phi^I\rangle$: the many-body embedding (impurity) wavefunction – likewise independent until its own Euler–Lagrange equation below forces it to be an eigenstate of $H_{\rm imp}$;
- $E_{\rm qp}$, $E^{I,c}$: Lagrange multipliers enforcing, respectively, $\langle\Psi_{\rm qp}|\Psi_{\rm qp}\rangle = 1$ and $\langle\Phi^I|\Phi^I\rangle = 1$;
- $\lambda^I, \lambda^{I,c} \in \mathbb{R}^{N_{\rm eff}\times N_{\rm eff}}$, $V^I \in \mathbb{R}^{N_{\rm eff}}$: Lagrange multipliers described below, which *after* extremization take on the roles of the onsite qp potential, the bath potential, and the hybridization entering $H_{\rm qp}/H_{\rm imp}$.

The Lagrangian. Adapting Eq. 29 of `NotesOngGut.pdf` to two real-space fragments coupled by hopping $-t$ (in place of a Brillouin-zone sum over a translationally invariant lattice):

$$\begin{aligned} L \;=\; & -t\sum_{I\neq J}(R^I)^T\langle\Psi_{\rm qp}|d_I^\dagger d_J|\Psi_{\rm qp}\rangle\, R^J \;+\; \sum_I\langle\Phi^I|H^I_{\rm loc}|\Phi^I\rangle \\ & +\; E_{\rm qp}\big(1-\langle\Psi_{\rm qp}|\Psi_{\rm qp}\rangle\big) \;+\; \sum_I E^{I,c}\big(1-\langle\Phi^I|\Phi^I\rangle\big) \\ & +\; \sum_I\Big\{(V^I)^T\Big[\langle\Phi^I|d_I^\dagger c_I|\Phi^I\rangle - \sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\,R^I\Big] + \text{h.c.}\Big\} \\ & +\; \sum_I \mathrm{Tr}\Big[\lambda^{I,c}\big(\Delta^{II} - \langle\Phi^I|d_I d_I^\dagger|\Phi^I\rangle\big)\Big] \;+\; \sum_I \mathrm{Tr}\Big[\lambda^I\big(\Delta^{II} - \langle\Psi_{\rm qp}|d_I^\dagger d_I|\Psi_{\rm qp}\rangle\big)\Big], \end{aligned} \tag{5}$$

where $\mathbf{I}$ is the $N_{\rm eff}\times N_{\rm eff}$ identity matrix and, exactly as in $H_{\rm imp}$ (Eq. (25)), the $\lambda^{I,c}$-term is written with the operator ordering $d_I d_I^\dagger$ (annihilation before creation), not $d_I^\dagger d_I$ – the two differ by the anticommutator identity of Eq. (28): using the "wrong" ordering here would reintroduce the sign bug already fixed in $H_{\rm imp}$ itself.

Role of each term.

1. *Kinetic term.* The bare quasiparticle hopping energy, evaluated on the as-yet-unconstrained wavefunction $|\Psi_{\rm qp}\rangle$. Once the Euler–Lagrange equation below is imposed, its expectation value becomes exactly the $-4t\,(R^0)^T\Delta^{01}R^1$ kinetic contribution to the final variational-energy formula, Eq. (39).
2. $\sum_I\langle\Phi^I|H^I_{\rm loc}|\Phi^I\rangle$. The physical interaction-plus-chemical-potential energy of each fragment's embedding wavefunction. Together with the kinetic term, and once every constraint below is satisfied, these give $E_{\rm var} = E^0_{\rm loc} + E^1_{\rm loc} - 4t(R^0)^T\Delta^{01}R^1$, Eq. (39).
3. $E_{\rm qp}$. Enforces normalization of $|\Psi_{\rm qp}\rangle$. Section 3.1 shows that extremizing L over $\langle\Psi_{\rm qp}|$ turns this multiplier into exactly the quasiparticle ground-state energy of $H_{\rm qp}$.
4. $E^{I,c}$. Enforces normalization of $|\Phi^I\rangle$. Extremizing L over $\langle\Phi^I|$ (Section 3.1) turns this multiplier into exactly the embedding ground-state energy $E^I_{\rm loc}$ that `impurity_solve` returns – unlike an earlier draft of this document, this multiplier is *not* inert: it is the eigenvalue of $H^I_{\rm imp}$ used throughout Section 6.3.

5. V^I*-term.* Enforces that the hybridization density $\langle\Phi^I|d_I^\dagger c_I|\Phi^I\rangle$ actually produced by the embedding wavefunction equals the qp-side amplitude $\sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\,R^I$ (the matrix square root converting a bath *occupation* into a coherent single-particle *amplitude*). This is the same relation as Eq. 16–17 of `NotesOngGut.pdf`. Stationarity with respect to V^I itself simply reinstates this constraint, which *is* the R-update, Eq. (36), of Section 6.4 (with $D^I = \langle\Phi^I|d_I^\dagger c_I|\Phi^I\rangle$); after extremization, V^I itself becomes the hybridization entering $H_{\rm imp}$.

6. $\lambda^{I,c}$*-term.* Enforces that the bath sector of the embedding wavefunction equals the shared variable Δ^{II}; after extremization, $\lambda^{I,c}$ becomes exactly the bath potential appearing in $H_{\rm imp}$, Eq. (25).

7. λ^I*-term.* Enforces that the 1-RDM block of the *quasiparticle* wavefunction, $\langle\Psi_{\rm qp}|d_I^\dagger d_I|\Psi_{\rm qp}\rangle$, also equals the same shared variable Δ^{II}; after extremization, λ^I becomes exactly the onsite potential entering $H_{\rm qp}$, Eq. (11). This is the fifth multiplier: it is easy to miss because, operationally (Section 7.1), the code never solves this constraint explicitly: λ^I is generated from the previous iteration's $\lambda^{I,c}$ through the stationarity condition of Eq. (20), and the equality of the quasiparticle block Δ^{II} (from the ground state of $H_{\rm qp}$) and the impurity-derived $\mathbf{I}-\Delta^{I,\rm bb}$ then holds only at self-consistency.

3.1 Euler–Lagrange equations

Variation with respect to $\langle\Psi_{\rm qp}|$. Only the kinetic term and the λ^I-term depend on $|\Psi_{\rm qp}\rangle$; setting $\delta L/\delta\langle\Psi_{\rm qp}| = 0$ gives

$$\Big[-t\sum_{I\neq J} R^I(R^J)^T d_I^\dagger d_J \;-\; \sum_I \lambda^I d_I^\dagger d_I\Big]|\Psi_{\rm qp}\rangle \;=\; E_{\rm qp}|\Psi_{\rm qp}\rangle, \qquad \text{i.e.} \qquad H_{\rm qp}|\Psi_{\rm qp}\rangle = E_{\rm qp}|\Psi_{\rm qp}\rangle, \quad (6)$$

recovering exactly the two-fragment quasiparticle Hamiltonian, Eq. (11): λ^I is confirmed to be a genuine onsite potential inside $H_{\rm qp}$, and $E_{\rm qp}$ is confirmed to be its ground-state energy.

Variation with respect to $\langle\Phi^I|$. Only $\langle\Phi^I|H^I_{\rm loc}|\Phi^I\rangle$, the V^I-term, and the $\lambda^{I,c}$-term depend on $|\Phi^I\rangle$; setting $\delta L/\delta\langle\Phi^I| = 0$ gives

$$\Big[H^I_{\rm loc}+\sum_a V^I_a\big(d^\dagger_{I,a}c_I + c_I^\dagger d_{I,a}\big) - \sum_{ab}\lambda^{I,c}_{ab}\,d_{I,b}d^\dagger_{I,a}\Big]|\Phi^I\rangle \;=\; E^{I,c}|\Phi^I\rangle, \qquad \text{i.e.} \qquad H^I_{\rm imp}|\Phi^I\rangle = E^I_{\rm loc}|\Phi^I\rangle, \quad (7)$$

recovering exactly the embedding Hamiltonian and its sign convention, both already fixed and validated in Eq. (25)/Eq. (28).

Variation with respect to R^I. R^I appears both in the kinetic term (as a coefficient of $\Delta^{IJ,(\rm qp)} \equiv \langle\Psi_{\rm qp}|d_I^\dagger d_J|\Psi_{\rm qp}\rangle$) and in the V^I-term; setting $\partial L/\partial R^I = 0$ gives

$$-2t\,\Delta^{IJ,(\rm qp)}R^J \;-\; \sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\,V^I \;=\; 0 \quad\Longrightarrow\quad \sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\,V^I \;=\; -2t\,\Delta^{IJ}R^J, \quad (8)$$

which is precisely the structure of the V-fit, Eq. (15) (the overall numerical prefactor of that equation, $-t$ rather than $-2t$, is a convention choice tied to how the fragment sum and the h.c. in Eq. (5) are counted, and is fixed, independently of this derivation, by the term-by-term validation against the reference implementation described in Section 6.1). This – not the V^I-term's own stationarity – is where the V-fit equation comes from; varying with respect to V^I itself, as noted above, gives back the R-update, Eq. (36), instead.

Where Eq. (20) comes from. Δ^{II} enters only the V^I-term, the $\lambda^{I,c}$-term, and the λ^I-term. Setting $\partial L/\partial\Delta^{II} = 0$, holding R^I, V^I *fixed* (i.e. without first re-expressing V^I through Eq. (15) as a function of Δ^{II}), gives

$$\lambda^I + \lambda^{I,c} \;=\; \frac{\partial}{\partial\Delta^{II}}\Big[(R^I)^T\sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}V^I\Big]\Big|_{\text{sym}}, \tag{9}$$

which is precisely Eq. (20) – the origin of the statement, made in Section 6.2, that λ^I and $\lambda^{I,c}$ are not independent variational parameters but two of the five Lagrange multipliers of Eq. (5), tied together by this single stationarity condition. Stationarity with respect to $\lambda^I, \lambda^{I,c}$ themselves simply reinstates the two density-matching constraints they were introduced to enforce.

4 General workflow: inputs, guesses, and order of computation

Sections 6 and 7 give the full, precise algebraic form of every equation the code evaluates. Before diving into that detail, this section is a bird's-eye map of the algorithm: what is *fixed* for the whole run, what is only ever a *guess*, and the exact *order* in which every other quantity is computed from those two. Every box below names the equation or section where it is defined in full; read this section first, then Sections 5–7 for the details it glosses over.

Relation to the reference C++ code. The cycle below is, step by step, the one executed by the reference C++ implementation (`GhostGutzwiller::Run` with the input flags `no_fit_scf T`, `gg_updater lin` and the `msite_ggm` matrix basis). Only the impurity problem is solved by a different code (the Jordan–Wigner exact diagonalization of Section 6.3). Every other operation – the regularized matrix square roots, the way λ is generated, the occupation rule of H_{qp}, the derivative entering λ^c, the energy formula, the mixing and the convergence test – is reproduced as in that code, and Section 7.4 shows the two codes agree iteration by iteration.

Three categories of quantity. Every symbol that appears anywhere in this document falls into exactly one of three categories:

1. **Fixed inputs** – set once, from the command line, and never changed by the algorithm itself: U, N_g (hence $N_{\text{eff}} = 1 + N_g$), t, the square-root shift ε (`--sqmat-shift`), the mixing weight a_{mix} and the number of substitution iterations n_{eq} (Section 7.2), the tolerances $\text{tol}_E, \text{tol}_{\text{mat}}$ and the maximum iteration count.

2. **A guess** – has a value only because *something* has to start the iteration. There is exactly one such object: the pair (R^I, λ^I) at iteration 0, coming either from `R.in`/`L.in` or from the default cold start (Section 10). From iteration 1 on, λ^I is *derived* (Eq. (40)) from quantities of the previous iteration; it is not fitted and not free.

3. **Derived quantities** – everything else (Δ^{II}, Δ^{IJ}, V^I, $\lambda^{I,c}$, $|\Phi^I\rangle$, E^I_{loc}, D^I, the updated R^I, Δ^{II}, and finally E_{var}) is computed, in a fixed order, from the current (R^I, λ^I) and the previous iteration's $(V^I, \lambda^{I,c})$. This order is the content of the flowchart below.

Order of computation, one outer iteration.

0. *Set* λ^I. At iteration 0 take the input guess; afterwards $\lambda^I = -\lambda^{I,c}_{\text{prev}} + G(\Delta^{II}, V^I_{\text{prev}}R^I)$ (Eq. (40)).

1. Diagonalize the quasiparticle Hamiltonian $H_{\mathrm{qp}}(R, \lambda)$ and fill its lowest N_{eff} levels to get Δ^{II}, Δ^{IJ} (Section 5.3).

2. Compute the hybridization V^I from R^J, Δ (Eq. (15)).

3. Compute the bath potential $\lambda^{I,c}$ from Δ, R^I, V^I and the *current* λ^I (Eq. (21)).

4. Diagonalize the embedding Hamiltonian $H_{\mathrm{imp}}(V, \lambda^c)$ to get $|\Phi^I\rangle$, its energy E^I_{loc}, and the 1-RDM blocks $D^I, \Delta^{I,\mathrm{bb}}$ (Section 6.3).

5. Update $\Delta^{II}_{\mathrm{new}} = \mathbf{I} - \Delta^{I,\mathrm{bb}}$ and R^I_{new} from $D^I, \Delta^{II}_{\mathrm{new}}$ (Eq. (36)); evaluate E_{var} (Eq. (39)).

6. Mix $(R^I, \Delta^{II}) \to (R^I_{\mathrm{next}}, \Delta^{II}_{\mathrm{next}})$ (Section 7.2), evaluate $|\Delta E|$, $\|\delta\Delta\|$, $\|\delta R\|$ and test convergence. If not converged, return to step 0 with the mixed quantities.

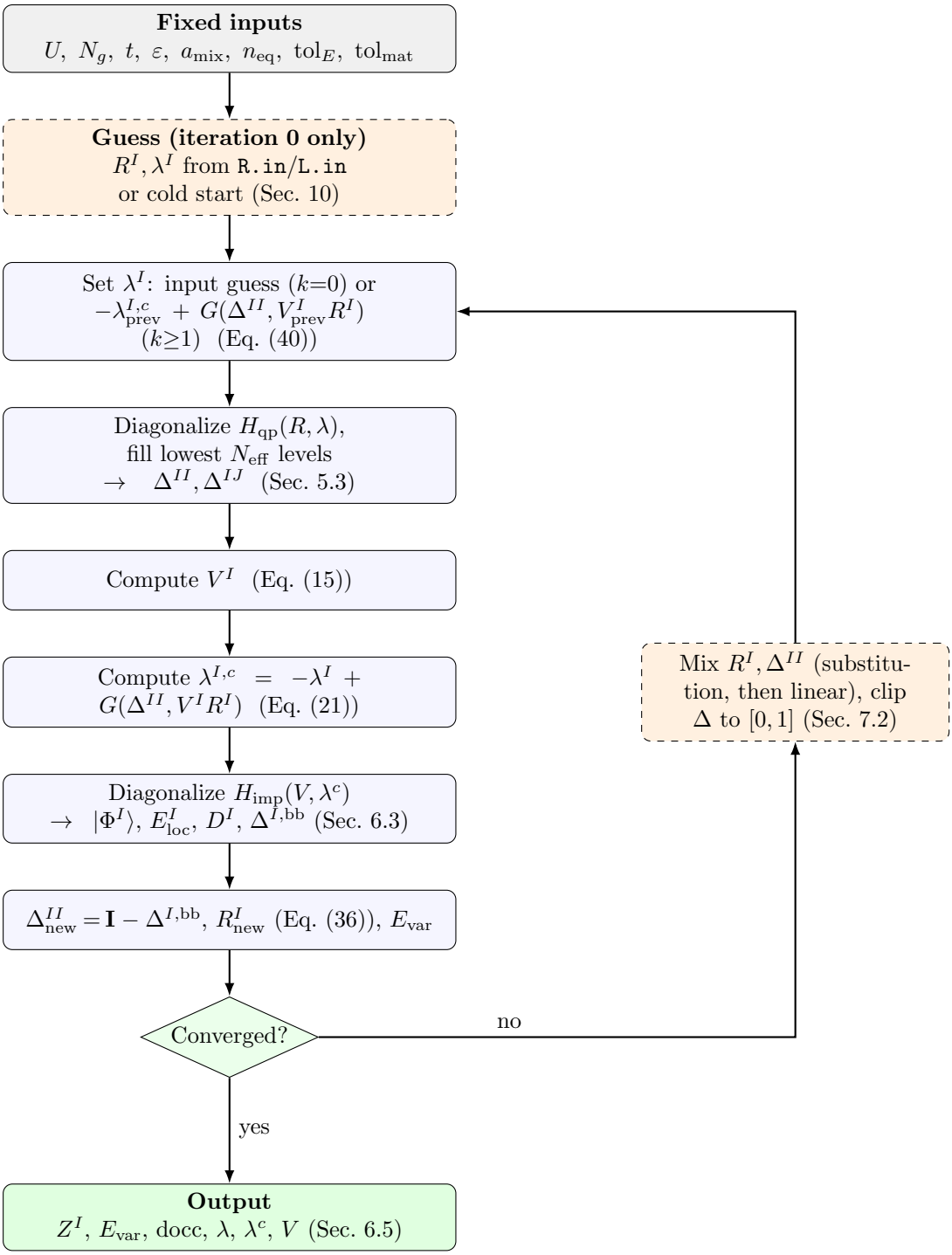


Figure 1: One outer self-consistency iteration. Gray = fixed for the whole run. Orange/dashed = a guess or the mixing step. Blue = quantities derived, in this order. Green = converged output. There is no root-finding step: at iterations $k \geq 1$ the potential λ^I is obtained by evaluating Eq. (40) at the mixed (R, Δ), and Δ^{II} is then *recomputed* from the quasiparticle ground state.

5 Specialization to the two-site, two-independent-fragment dimer

5.1 Each site is its own fragment; the two fragments need not be equal

Each site is treated as its own impurity/fragment. This is the standard gGA lattice construction applied to a finite two-site "lattice": lumping the whole dimer into a single impurity would leave no inter-site hopping to renormalize and would make the method vacuous.

Important correction relative to an earlier version of this project. It is tempting to assume that, because the physical Hamiltonian Eq. (1) is symmetric under exchanging sites $0 \leftrightarrow 1$, the two fragments' variational parameters must be equal ($R^0 = R^1$, $\lambda^0 = \lambda^1$), so that only one impurity problem needs to be solved per iteration. This is *not* true in general. A reference calculation for this same dimer at $U/t = 2$, $N_g = 2$ converges to a *genuinely asymmetric* fixed point, with numerically distinct $R^0 \neq R^1$ and $\lambda^0 \neq \lambda^1$ (even though $|R^0| \approx |R^1|$, their components differ substantially – see Table 3). The physical dimer's site-exchange symmetry can be *spontaneously broken* at the level of the auxiliary variational parameters, exactly as in a mean-field theory with more than one self-consistent solution. The code therefore **always** treats the two fragments as independent variational objects; the possibility that a converged solution happens to satisfy $R^0 = R^1$ is a property of the solution, never an assumption imposed on the algorithm.

5.2 Quasiparticle Hamiltonian for two independent fragments

Let $R^0, R^1 \in \mathbb{R}^{N_{\text{eff}}}$ (column vectors, since $N_{\text{phys}} = 1$) be the two fragments' renormalization vectors, and $\lambda^0, \lambda^1 \in \mathbb{R}^{N_{\text{eff}} \times N_{\text{eff}}}$ (real symmetric) their onsite potentials. Because $N_{\text{phys}} = 1$, the fully general multi-orbital quasiparticle Hamiltonian

$$H_{\text{qp}} \;=\; \sum_{I \neq J} \sum_{a,b} \Big(\sum_{\alpha,\beta} R^{I\dagger}_{a\alpha}\, t^{\alpha\beta}_{IJ}\, R^{J}_{\beta b} - \delta_{IJ} \lambda^{I}_{ab} \Big) d^{\dagger}_{Ia} d_{Jb} \tag{10}$$

collapses, for our two fragments $I, J \in \{0, 1\}$ and scalar hopping $t_{01} = t_{10} = -t$ (with $t^{\alpha\beta}_{IJ}$ having only one value of α, β), to the explicit $2N_{\text{eff}} \times 2N_{\text{eff}}$ real-space matrix (per spin channel; both spin channels use the same matrix)

$$\boxed{ H_{\text{qp}} \;=\; \begin{pmatrix} -\lambda^0 & -t\, R^0 (R^1)^T \\ -t\, R^1 (R^0)^T & -\lambda^1 \end{pmatrix} } \tag{11}$$

This is the direct generalization of the single-fragment (symmetric) block matrix $H_{\text{qp}} = \left[\begin{smallmatrix} -\lambda & -tRR^T \\ -tRR^T & -\lambda \end{smallmatrix}\right]$ to the case $R^0 \neq R^1$, $\lambda^0 \neq \lambda^1$; it reduces to the symmetric form exactly when $R^0 = R^1 = R$ and $\lambda^0 = \lambda^1 = \lambda$. Because there are only two real-space sites, H_{qp} is diagonalized directly (no Brillouin-zone integral is needed, unlike the periodic-lattice formulation this is specialized from).

Code: `qp_state(R, lam, t)` builds exactly Eq. (11) and diagonalizes it with `numpy.linalg.eigh`; it returns H_{qp} and the ground-state 1-RDM ρ^{qp} of Section 5.3.

5.3 Quasiparticle ground state and occupation rule

H_{qp} has $2N_{\text{eff}}$ single-particle levels per spin. The quasiparticle ground state fills a *fixed number* of them. In the reference code the molecular class (`RSImpMolecule`, used for `mol_type rsimp_molecule`) occupies, per spin, the lowest

$$n_{\text{eff}} \;=\; n_{\text{ph}} + \frac{N_{\text{eff,tot}} - N_{\text{phys,tot}}}{2} \;=\; 1 + \frac{2N_{\text{eff}} - 2}{2} \;=\; N_{\text{eff}} \tag{12}$$

eigenvectors of $H_{\rm qp}$ (`mol_nups`= $n_{\rm ph} = 1$; $N_{\rm eff,tot} = 2N_{\rm eff}$ quasiparticle orbitals and $N_{\rm phys,tot} = 2$ physical orbitals for the two fragments). With eigenvalues $\varepsilon_1 \le \cdots \le \varepsilon_{2N_{\rm eff}}$ and orthonormal eigenvectors $|n\rangle$,

$$\rho^{\rm qp} \;=\; \sum_{n=1}^{N_{\rm eff}} |n\rangle\langle n|, \tag{13}$$

a fixed *level-count* rule: it does not look at the signs of the ε_n and there is no adjustable chemical potential. The particle-hole symmetric half filling of the physical dimer enters through the constant $-\mu = -U/2$ contained in $H_{\rm loc}$ (Section 6.3).

Caution: the base-class rule is different. The parent class in the reference code (`RSImpMod`, used by non-molecular impurity models) uses instead a zero-temperature Fermi function at chemical potential 0 (occupy $\varepsilon_n < 0$). The two rules coincide whenever $H_{\rm qp}$ has exactly $N_{\rm eff}$ negative eigenvalues, which is the case at the seeds and at the converged solutions of the near-converged test cases of Section 7.4 (there the spectrum is symmetric about zero, e.g. $\pm 0.0197, \pm 0.9375, \pm 0.9862$ for the symmetric $U/t = 2$ solution), so those cases cannot tell them apart. Away from self-consistency they do differ: for example the cold seed $R = (1, 1, 1)$, $\lambda \approx 0$ has eigenvalues $-2.9995, -0.0002, +7\times 10^{-6}, \ldots$ with only two negative ones, and the two rules then give different Δ^{11}. The code follows the molecular class, i.e. Eq. (13). (An intermediate revision of the code used the sign rule by mistake, from reading the base class; it was found by comparing iteration 0 of a cold start with the reference.)

$\rho^{\rm qp}$ is a $2N_{\rm eff} \times 2N_{\rm eff}$ matrix; its blocks give the three independent quasiparticle 1-RDM blocks needed below:

$$\Delta^{00} = \rho^{\rm qp}_{[0,N_{\rm eff})\times[0,N_{\rm eff})}, \qquad \Delta^{11} = \rho^{\rm qp}_{[N_{\rm eff},2N_{\rm eff})\times[N_{\rm eff},2N_{\rm eff})}, \qquad \Delta^{01} = \rho^{\rm qp}_{[0,N_{\rm eff})\times[N_{\rm eff},2N_{\rm eff})}. \tag{14}$$

Δ^{00} and Δ^{11} are symmetric; Δ^{01} is in general *not* symmetric, and $\Delta^{10} = (\Delta^{01})^T$. In the text Δ^{II} denotes the fragment-diagonal blocks Δ^{00}, Δ^{11} and Δ^{IJ} the off-diagonal one.

Code: in `qp_state`, `occ = np.zeros(2*Neff); occ[:Neff] = 1` implements Eq. (13) (`eigh` returns ascending eigenvalues), and `rho = (evecs * occ) @ evecs.T`. The blocks of Eq. (14) are slices of `rho` (`blocks(rho)` in `run_gga`, and inside `hybridization_V`).

6 Self-consistency equations

The gGA self-consistency loop for two fragments couples the quasiparticle side (Section 5) to an embedding/impurity side through four fitting/update relations, specialized here from the general multi-fragment formulation (Mejuto-Zaera, arXiv:2403.05157, Eqs. 5–11) and validated numerically, block by block, against an independent reference implementation (see Table 3).

6.1 Hybridization V (Eq. 7 of the reference)

For fragment I, the hybridization vector $V^I \in \mathbb{R}^{N_{\rm eff}}$ is fixed by requiring

$$\sqrt{\Delta^{II}\big(\mathbf{I} - \Delta^{II}\big)}\; V^I \;=\; -t\, \Delta^{IJ}\, R^J, \qquad J \neq I, \tag{15}$$

i.e. for our two fragments,

$$\sqrt{\Delta^{00}(\mathbf{I} - \Delta^{00})}\; V^0 = -t\, \Delta^{01} R^1, \tag{16}$$

$$\sqrt{\Delta^{11}(\mathbf{I} - \Delta^{11})}\; V^1 = -t\, (\Delta^{01})^T R^0. \tag{17}$$

Notice that V^I is fit using the *neighbor* fragment's R^J (not the fragment's own R^I), and the *cross*-fragment block Δ^{IJ} (not Δ^{II}). Getting this backwards (using R^I instead of R^J) is invisible in the symmetric special case $R^0 = R^1$ but produces a completely different, wrong answer once the two fragments differ – this was confirmed empirically while developing the code.

The matrix square root in Eq. (15) is the *shifted* one used by the reference code,

$$S(\Delta) \;=\; \sqrt{\Delta(\mathbf{I}-\Delta)+\varepsilon\,\mathbf{I}}, \qquad \varepsilon = 10^{-9}\ \text{(default)}, \tag{18}$$

evaluated in the eigenbasis of Δ^{II}: if $\Delta^{II} = W\,\mathrm{diag}(n_1,\ldots,n_{N_{\mathrm{eff}}})\,W^T$, then $S(\Delta^{II}) = W\,\mathrm{diag}(s_a)\,W^T$ with $s_a = \sqrt{n_a(1-n_a)+\varepsilon}$, and Eq. (15) is solved eigenvalue by eigenvalue,

$$V^I \;=\; W\,\mathrm{diag}\!\left(\frac{1}{s_a}\right) W^T\left(-t\,\Delta^{IJ}R^J\right). \tag{19}$$

The shift ε. At the self-consistent solutions one or more eigenvalues of Δ^{II} sit at 0 or 1 (a ghost bath orbital completely empty or completely full). For the symmetric $U/t = 2$, $N_g = 2$ solution the eigenvalues of Δ^{00} are 0, $\frac{1}{2}$, 1 to machine precision, so $n_a(1-n_a)$ is 0 or a rounding-level number (of either sign), and the unshifted $1/\sqrt{n_a(1-n_a)}$ is infinite or undefined; since the corresponding component of the right-hand side is also numerically zero, the raw formula produces 0/0 and the whole iteration becomes NaN (this is what the earlier version of the code did on that case). The reference code avoids this by adding the shift ε (input key `gg_sqmat_shift`) to the *eigenvalues* of $\Delta(\mathbf{I}-\Delta)$ before taking the square root (its `GetMatSqrt(A, regul)` returns $\sqrt{\text{eigenvalue}+\text{regul}}$) and then solving the linear system with that matrix; Δ itself is *not* clipped beforehand. The code does the same. A component along a direction with $n_a \approx 0, 1$ is then $\text{rhs}/\sqrt{\varepsilon} \sim 3\times10^4\,\text{rhs}$ with rhs itself vanishingly small, hence finite, and the value agrees with the reference. The eigenvalue clip (10^{-10}) and the output cap ($V_{\max} = 10^2$) of the earlier version have been removed: they are not part of the reference algorithm.

Code: `hybridization_V(rho, R, t, shift, i)` forms the right-hand side $-t\,\rho_{ij}R^J$ from the blocks of ρ^{qp} (for fragment 0: $\Delta^{01}R^1$; for fragment 1: $\Delta^{10}R^0 = (\Delta^{01})^T R^0$, matching Eq. (15) exactly, with the neighbor's R) and applies $S(\Delta^{II})^{-1}$ through `apply_S_inv(Delta, B, shift)`.

6.2 Bath potential λ^c (Eq. 8 of the reference)

The onsite potential λ^I that enters H_{qp} (Eq. (11)) and the bath potential $\lambda^{I,c}$ that enters the impurity Hamiltonian (Section 6.3) are *not* independent variational parameters. They arise as Lagrange multipliers in the underlying constrained variational problem, and stationarity of the Lagrangian with respect to the quasiparticle density matrix $\Delta^{(\mathrm{qp})}$ gives the linear relation

$$\lambda^I_{ab} + \lambda^{I,c}_{ab} \;=\; \frac{\partial}{\partial\Delta^{II}_{ab}}\Big[R^I\cdot\sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\cdot V^I\Big]\Big|_{\mathrm{sym}}, \tag{20}$$

where R^I and V^I are held *fixed* in the derivative (i.e. V^I is *not* re-substituted through Eq. (15) as a function of Δ^{II} before differentiating), and "sym" denotes the standard symmetric-matrix-gradient convention: for $a = b$ the derivative is taken with respect to the single diagonal entry, while for $a \neq b$ it is the *sum* of the two formally independent partial derivatives with respect to Δ_{ab} and Δ_{ba} (equivalently: differentiate along the symmetric perturbation direction $X_{ab} + X_{ba}$, not along X_{ab} alone, where X_{ab} denotes the elementary matrix with a 1 in position (a,b) and 0 elsewhere – denoted X rather than the more common E to avoid any visual clash with the energies E_{qp}, $E^{I,c}$,

$E_{\rm loc}$, $E_{\rm var}$ used throughout this document). Equation (20) is used to *define* the bath potential from the current λ^I:

$$\lambda^{I,c} \;=\; -\lambda^I + G\big(\Delta^{II},\, V^I R^I\big), \tag{21}$$

where V^I is a column, R^I a row, $M = V^I R^I$ is an $N_{\rm eff} \times N_{\rm eff}$ matrix, and $G(\Delta, M)$ is the symmetric derivative of the scalar $S_M(\Delta) = \mathrm{Tr}\big[M\, S(\Delta)\big] = R\, S(\Delta)\, V$ with respect to Δ (with R, V held fixed), in the convention of the reference code. That convention is: expand in an orthonormal basis $\{B_k\}$ of the real symmetric $N_{\rm eff} \times N_{\rm eff}$ matrices (the generalized Gell-Mann basis, `msite_ggm`; it contains the identity, so it is complete), take the directional derivative $D_k = \mathrm{d}S[B_k]$ of the matrix square root, and set

$$G(\Delta, M) \;=\; \sum_k B_k \,\Big(\mathrm{Tr}[M D_k] + \mathrm{Tr}[D_k^T M^T]\Big) \;=\; \sum_k B_k \; 2\,\mathrm{Tr}[M D_k], \tag{22}$$

(the two traces are equal because D_k is symmetric). Since the basis is orthonormal, G is twice the projection of the gradient of S_M onto the symmetric matrices; the factor 2 is the "diagonal doubling" that had to be found empirically in an earlier version of the code.

Analytic evaluation of G. The derivative of the shifted square root is available in closed form in the eigenbasis of Δ. Let $\Delta = W\,\mathrm{diag}(n_a)\,W^T$, $s_a = \sqrt{n_a(1-n_a)+\varepsilon}$, and use a tilde for quantities in this eigenbasis, $\tilde{X} = W^T X W$. For a symmetric perturbation B, the derivative of $P = \Delta(\mathbf{I}-\Delta)$ is $\mathrm{d}P = B - B\Delta - \Delta B$, i.e. $\widetilde{\mathrm{d}P}_{ab} = \tilde{B}_{ab}\,(1 - n_a - n_b)$. Differentiating $S^2 = P + \varepsilon\mathbf{I}$ gives the Sylvester equation $S\,\mathrm{d}S + \mathrm{d}S\,S = \mathrm{d}P$, which is diagonal in this basis:

$$\widetilde{\mathrm{d}S}_{ab} \;=\; w_{ab}\,\tilde{B}_{ab}, \qquad w_{ab} \;=\; \frac{1 - n_a - n_b}{s_a + s_b}. \tag{23}$$

Then $\mathrm{Tr}[M\,\mathrm{d}S[B]] = \sum_{ab} \tilde{M}_{ba}\, w_{ab}\, \tilde{B}_{ab}$, and Eq. (22) collapses (summing over the complete orthonormal basis gives the symmetric part of the coefficient matrix) to

$$\boxed{\; G(\Delta, M) \;=\; W\Big[\big(\tilde{M} + \tilde{M}^T\big) \circ w\Big] W^T \;} \tag{24}$$

where $\circ$ is the elementwise product. This is exact: it replaces the central finite-difference evaluation of the earlier version (step $h = 10^{-8}$ on $\sqrt{\Delta(\mathbf{I}-\Delta)}$ through `scipy.linalg.sqrtm`), which is unreliable precisely where it matters, namely when Δ has eigenvalues at 0 or 1 and S is not differentiable in the unshifted problem. With the shift, w_{ab} stays finite for all a, b.

Code: `grad_S(Delta, M, shift)` implements Eq. (24); `embedding_lambda_c(Delta, lam, V, R, shift)` returns $-\lambda + G(\Delta, VR)$, i.e. Eq. (21).

6.3 Impurity (embedding) Hamiltonian

For each fragment I, the embedding Hamiltonian couples the physical, interacting orbital (both spins, operators $c_\uparrow, c_\downarrow$) to $N_{\rm eff}$ non-interacting bath orbitals per spin (operators $d_{a\uparrow}, d_{a\downarrow}$, $a = 1, \ldots, N_{\rm eff}$):

$$H^I_{\rm imp} \;=\; H_{\rm loc} \;+\; \sum_{a=1}^{N_{\rm eff}} \sum_\sigma V_a^I \big(d^\dagger_{a\sigma} c_\sigma + c^\dagger_\sigma d_{a\sigma}\big) \;-\; \sum_{a,b=1}^{N_{\rm eff}} \sum_\sigma \lambda^{I,c}_{ab}\, d_{b\sigma} d^\dagger_{a\sigma}, \tag{25}$$

with the local (physical) Hamiltonian, using the *same* U and particle-hole-symmetric $\mu = U/2$ as the physical dimer,

$$H_{\rm loc} \;=\; U\, n_\uparrow n_\downarrow \;-\; \mu\,(n_\uparrow + n_\downarrow), \qquad n_\sigma = c^\dagger_\sigma c_\sigma. \tag{26}$$

Sign of the bath-potential term – a subtlety. Equation (25)'s bath term is written, as it appears in the underlying Lagrangian, as $-\lambda^{I,c}_{ab}\, d_{b\sigma} d^\dagger_{a\sigma}$ (annihilation operator first). This is *not* the same as $-\lambda^{I,c}_{ab}\, d^\dagger_{a\sigma} d_{b\sigma}$: using the fermionic anticommutator $\{d_{a\sigma}, d^\dagger_{b\sigma}\} = \delta_{ab}$,

$$d_{b\sigma} d^\dagger_{a\sigma} \;=\; \delta_{ab} - d^\dagger_{a\sigma} d_{b\sigma}, \tag{27}$$

so, dropping the physically irrelevant additive constant $-\sum_{a\sigma} \lambda^{I,c}_{aa}$,

$$-\sum_{ab\sigma} \lambda^{I,c}_{ab}\, d_{b\sigma} d^\dagger_{a\sigma} \;\widehat{=}\; +\sum_{ab\sigma} \lambda^{I,c}_{ab}\, d^\dagger_{a\sigma} d_{b\sigma}. \tag{28}$$

The code implements the *right-hand* (normal-ordered, + sign) form of Eq. (28). Implementing the naive left-hand form with a literal − sign in front of $d^\dagger_a d_b$ is a sign error that was present in an earlier version of the code and produced impurity ground-state energies and 1-RDMs that disagreed with an independent reference implementation by an order of magnitude; it was found and fixed by comparing intermediate quantities (bath–bath and bath–impurity blocks of the impurity 1-RDM) term by term against a reference calculation.

Exact diagonalization via Jordan–Wigner strings. The impurity Hamiltonian, Eq. (25), is diagonalized exactly, in the half-filled sector of the Fock space (see "Ground-state sector" below), using explicit Jordan–Wigner fermion operators stored as sparse matrices. For n_{modes} ordered fermionic modes, the annihilation operator for mode i is

$$c_i \;=\; Z^{\otimes i} \otimes c_- \otimes \mathbf{I}^{\otimes(n_{\mathrm{modes}}-i-1)}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad c_- = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \tag{29}$$

acting on the $2^{n_{\mathrm{modes}}}$-dimensional local Fock space, with the local basis convention $c_-|1\rangle = |0\rangle$, $c_-|0\rangle = 0$ (i.e. $(0,1)^T$ represents the occupied state). The mode ordering used for the impurity problem is

$$0:\; c_\uparrow, \quad 1:\; c_\downarrow, \quad 2{+}2k:\; d_{k\uparrow}, \quad 3{+}2k:\; d_{k\downarrow} \qquad (k = 0, \ldots, N_{\mathrm{eff}} - 1), \tag{30}$$

giving $n_{\mathrm{modes}} = 2(1 + N_{\mathrm{eff}})$ total modes and confirming Eq. (4)'s Fock dimension $2^{2(1+N_{\mathrm{eff}})} = 4^{1+N_{\mathrm{eff}}} = 4^{2+N_g}$. `exact_dimer.py` uses the identical Jordan–Wigner construction, Eq. (29), for the four physical modes of the full dimer.

Precomputation. All operator products that do not depend on V^I or $\lambda^{I,c}$ (namely $n_\uparrow$, $n_\downarrow$, H_{loc}, the hybridization bilinears $d^\dagger_{a\sigma} c_\sigma + \mathrm{h.c.}$, and the bath–bath bilinears $d^\dagger_{a\sigma} d_{b\sigma}$) are built *once* per (N_{eff}, U) pair (function `build_impurity_ops`), not rebuilt at every self-consistency iteration; the impurity Hamiltonian for a given iteration is then simply the weighted sum $H_{\mathrm{imp}} = H_{\mathrm{loc}} + \sum_a V_a\,(\text{hyb bilinear})_a + \sum_{ab} \lambda^c_{ab}\,(\text{bath bilinear})_{ab}$. This changes the per-iteration cost from rebuilding N^2_{eff} operator products of size dim × dim every iteration, with dim $= 4^{1+N_{\mathrm{eff}}}$, to a single diagonalization of the sector block (dimension 36 for $N_g = 2$, 400 for $N_g = 4$, see below); it was the dominant bottleneck before being fixed. The operators are stored as `scipy.sparse` matrices: they have only a few non-zeros per row, whereas dense dim × dim matrices for $N_g = 4$ (dim = 4096, 134 MB each, about 80 of them) would need ~ 10 GB.

Ground-state sector. The reference solver (`CMZEDsolver`) does not search the whole Fock space. For an even number of impurity orbitals per spin, $1+N_{\text{eff}} = 2+N_g$ (this is why N_g must be even), it diagonalizes H_{imp} only in the half-filled, spin-balanced sector $N_\uparrow = N_\downarrow = (1+N_{\text{eff}})/2$, of dimension $\binom{1+N_{\text{eff}}}{(1+N_{\text{eff}})/2}^2$ (36 for $N_g = 2$, instead of 256). The code does the same: `build_impurity_ops` stores the index set `sector` of basis states with these occupations (mode j is occupied in basis state i iff bit $n_{\text{modes}} - 1 - j$ of i is set, for the Jordan–Wigner ordering above) and `impurity_solve` diagonalizes $H_{\text{imp}}[\texttt{sector}, \texttt{sector}]$, then embeds the lowest eigenvector back into the full space so that all expectation values below are unchanged. This matters far from self-consistency: with the $\lambda^{I,c}$ of an early cold-start iteration the *global* lowest state of H_{imp} can lie in another sector (a triplet, or a different particle number), with a gap to the half-filled singlet of only $\sim 10^{-3}$, and the two solvers then return different $\Delta^{I,\text{bb}}$ and D^I. Near convergence the global ground state is in the sector and the restriction changes nothing.

Hand-over through an FCIDUMP text file. The reference code does not pass V^I and $\lambda^{I,c}$ to its solver directly: it writes the embedding Hamiltonian to a text `FCIDUMP` file (`WriteImpFCIDUMP`), each element printed with `std::scientific` at precision 12 (i.e. `%.12e`), and *omits* every element with $|x| < 10^{-10}$. The solver therefore sees V^I and $\lambda^{I,c}$ quantized this way. The code reproduces the quantization (function `fcidump_round`, applied to V^I and $\lambda^{I,c}$ before `impurity_solve`; disable with `--no-fcidump-rounding`). It is not cosmetic. The R update $R = S(\Delta_{\text{new}})^{-1}D$ divides by $s_a = \sqrt{n_a(1-n_a)+\varepsilon}$, and at the cold seeds Δ_{new} has eigenvalues within 10^{-8}–10^{-16} of 1, so noise well below 10^{-10} is amplified into $O(1)$ changes of R. Without the quantization, perturbing a seed by 10^{-12} flips the run between two different fixed points (Section 7.4); with it the trajectory is a deterministic function of the seed, as it is in the reference.

Observables. Given the impurity ground state $|\psi\rangle$ (lowest eigenvector of H_{imp} in the sector above), all needed expectation values are computed via matrix–vector products only (never forming $O(\dim \times \dim)$ operator products explicitly, for the same performance reason):

$$\Delta^{I,\text{bb}}_{ab} = \tfrac{1}{2}\Big(\langle\psi|d^\dagger_{a\uparrow}d_{b\uparrow}|\psi\rangle + \langle\psi|d^\dagger_{a\downarrow}d_{b\downarrow}|\psi\rangle\Big), \tag{31}$$

$$D^I_a = \tfrac{1}{2}\Big(\langle\psi|d^\dagger_{a\uparrow}c_{\uparrow}|\psi\rangle + \langle\psi|d^\dagger_{a\downarrow}c_{\downarrow}|\psi\rangle\Big), \tag{32}$$

$$E^I_{\text{loc}} = \langle\psi|H_{\text{loc}}|\psi\rangle, \qquad \text{docc}^I = \langle\psi|n_\uparrow n_\downarrow|\psi\rangle, \qquad n^I_{\text{phys}} = \langle\psi|(n_\uparrow+n_\downarrow)|\psi\rangle. \tag{33}$$

The spin-average in $\Delta^{I,\text{bb}}$ and D^I assumes a paramagnetic (spin-unpolarized) solution, which is consistent with the fact that V^I and $\lambda^{I,c}$ enter H_{imp} identically for both spin channels.

Code: `build_impurity_ops(Neff, U, mu)` and `impurity_solve(V, lam_c, ops)`.

How D^I is actually evaluated. $D^I_a = \langle\psi|d^\dagger_{a\sigma}c_\sigma|\psi\rangle$ is a *mixed* bath–impurity expectation value, but it is never evaluated by forming the $\dim \times \dim$ operator product $d^\dagger_a c$ itself. Instead, since d_a and c are already stored as the elementary annihilation-operator matrices from Eq. (29), each is applied *once* to the ground-state vector $|\psi\rangle$, and the two resulting vectors are combined with a single inner product:

$$|u_a\rangle \equiv d_{a\uparrow}|\psi\rangle, \qquad |v\rangle \equiv c_\uparrow|\psi\rangle, \qquad \langle\psi|d^\dagger_{a\uparrow}c_\uparrow|\psi\rangle \;=\; \langle u_a|v\rangle, \tag{34}$$

and likewise for the $\downarrow$ spin channel, with D^I_a the real part of the spin average of the two. The dagger on d_a is never applied explicitly: it falls out for free from the conjugate-linearity of the inner product's first argument (`numpy.vdot(x, y) = x.conj() @ y`), so $\langle d_{a\uparrow}\psi|c_\uparrow\psi\rangle = \langle\psi|d^\dagger_{a\uparrow}c_\uparrow|\psi\rangle$ comes

out automatically. The bath–bath block $\Delta^{I,\mathrm{bb}}_{ab}$ just above is evaluated the same way, as $\langle u_a|u_b\rangle$ using the same precomputed $|u_a\rangle$ vectors. This reduces the cost of computing every observable in Eq. (35)–above from $O(N_{\mathrm{eff}}^2 \cdot \dim^2)$ (building each mixed operator explicitly, then a matrix–vector product) to $O(N_{\mathrm{eff}} \cdot \dim)$ for the N_{eff} vector applications plus $O(N_{\mathrm{eff}}^2)$ cheap inner products – the same precomputation-driven cost reduction already noted above for H_{imp} itself.

Code: the vectors `bu_gs[a]`, `bd_gs[a]`, `cpu_gs`, `cpd_gs` in `impurity_solve` (`gga_dimer.py:591--594`) are exactly $|u_a\rangle$ (both spins) and $|v\rangle$ (both spins) above; `D[a] = 0.5*(np.vdot(bu_gs[a], cpu_gs) + np.vdot(bd_gs[a], cpd_gs)).real` (`gga_dimer.py:603--607`) implements the formula directly.

6.4 Update of R and Δ (Eq. 10 of the reference)

From the impurity ground state's bath–bath and bath–impurity 1-RDM blocks, each fragment's target quasiparticle quantities are updated as

$$\Delta^{II,\,\mathrm{new}} = \mathbf{I} - \Delta^{I,\mathrm{bb}}, \tag{35}$$

$$\sqrt{\Delta^{II,\mathrm{new}}\big(\mathbf{I} - \Delta^{II,\mathrm{new}}\big)}\; R^{I,\mathrm{new}} = D^I. \tag{36}$$

Equation (36) is solved exactly as Eq. (15), in the eigenbasis of $\Delta^{II,\mathrm{new}}$, with the same shifted square root $S(\Delta^{II,\mathrm{new}})$ of Eq. (18), i.e. $R^{I,\mathrm{new}} = S(\Delta^{II,\mathrm{new}})^{-1} D^I$.

Code: `apply_S_inv(Delta_new, D, shift)`.

Where Eq. (36) comes from. Equation (36) is not a new, independent equation – it is the mirror image of the V-fit, Eq. (15) (Section 6.1). Recall the coupling term of the Lagrangian, Eq. (5):

$$(V^I)^T\Big[\langle\Phi^I|d_I^\dagger c_I|\Phi^I\rangle - \sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\,R^I\Big] \;+\; \text{h.c.} \tag{37}$$

V^I sits there purely as a Lagrange multiplier enforcing that the bracket vanish. Varying the Lagrangian with respect to V^I itself does not fix a new value for V^I; it simply reinstates the constraint the bracket represents:

$$\langle\Phi^I|d_I^\dagger c_I|\Phi^I\rangle \;=\; \sqrt{\Delta^{II}(\mathbf{I}-\Delta^{II})}\,R^I, \tag{38}$$

which is exactly Eq. (36) once the left-hand side is identified with D^I (Section 3.1, "Variation with respect to R^I" explicitly notes that varying with respect to V^I, rather than R^I, gives back this R-update instead of the V-fit). So Eqs. (15) and (36) are the *same* underlying relation $\sqrt{\Delta(\mathbf{I}-\Delta)}\,R = D$-type constraint, solved for different unknowns at different points of the outer iteration: Eq. (15) solves it for V given R, Δ from the qp side, while Eq. (36) solves it for R given D, Δ freshly produced by the impurity solve.

The $\sqrt{\Delta(\mathbf{I}-\Delta)}$ factor is a coherence weight per bath direction: an orbital that is essentially empty ($\Delta_{aa} \to 0$) or essentially full ($\Delta_{aa} \to 1$) has no partial occupation left to fluctuate coherently with the physical orbital, so the factor vanishes and that direction cannot support any R component no matter how large D_a is – exactly the singular case handled by the shift ε of Eq. (18). A half-filled orbital ($\Delta_{aa} = 0.5$) has the maximal factor 0.5 and can carry the most weight. Eq. (36) is thus best read as: the updated renormalization amplitude R^I is the mixed density D^I, rescaled per bath direction by how much coherent single-particle weight is actually available in that direction.

6.5 Variational energy

The total variational energy of the two-site lattice, per the general formula $E_0 = \sum_I E^I_{\rm imp} + \sum_{I\neq J}\sum_{ab}(R^{I\dagger}t_{IJ}R^J)_{ab}\Delta^{({\rm qp})}_{ab}$ specialized to $N_{\rm phys}=1$, $t_{01}=t_{10}=-t$, and summing the ordered pair $(0,1)$ and $(1,0)$ contributions (which are equal, since each is the transpose of a scalar), reads

$$\boxed{E_{\rm var} \;=\; E^0_{\rm loc} + E^1_{\rm loc} \;-\; 4t\,\big(R^0\big)^T\Delta^{01}R^1.} \tag{39}$$

The factor of 4 combines: a factor 2 from summing the two ordered fragment pairs $(I,J)=(0,1)$ and $(1,0)$ (the physical hopping term already counts the bond once via $c_0^\dagger c_1 + {\rm h.c.}$, and the ordered-pair sum over $I\neq J$ reproduces exactly this without double counting), and another factor 2 from summing the two spin channels (which contribute identically under the paramagnetic assumption). $E^0_{\rm loc}+E^1_{\rm loc}$ (not $2E_{\rm loc}$) is used because the two fragments' impurity ground states are, in general, different.

At $U=0$, for any N_g, the code reproduces $E_{\rm var}=-2t$ exactly (the exact, uncorrelated dimer answer), which is used throughout as a hard consistency check on Eq. (39) and on every other formula that feeds into it.

7 Numerical solution of the self-consistency loop

Sections 6.1–6.4 define, for a *given* $(R^0,R^1,\lambda^0,\lambda^1)$, how to compute $\Delta^{00},\Delta^{11},\Delta^{01}$ (Section 5), V^0,V^1 (Section 6.1), $\lambda^{0,c},\lambda^{1,c}$ (Section 6.2), the impurity solve (Section 6.3), and the updated $R^I_{\rm new},\Delta^{II}_{\rm new}$ (Section 6.4). What remains is (a) how λ^I is produced at each outer iteration, and (b) how the iteration over (R,Δ) is driven to a fixed point. As stated in Section 4, both follow the reference C++ code exactly.

7.1 Where λ^I comes from: no fit, just Eq. (20)

At iteration 0, λ^I is the input guess and Δ^{II} is simply the block of the quasiparticle ground state $\rho^{\rm qp}(R,\lambda)$ built from it. At every later iteration $k\geq 1$, λ^I is obtained by solving the stationarity condition Eq. (20) for λ^I, using the previous iteration's bath potential and hybridization and the current (mixed) R^I and Δ^{II}:

$$\lambda^I_k \;=\; -\lambda^{I,c}_{k-1} \;+\; G\big(\Delta^{II}_k,\, V^I_{k-1}R^I_k\big). \tag{40}$$

Then $H_{\rm qp}(R_k,\lambda_k)$ is diagonalized and $\Delta^{II}_k,\Delta^{IJ}_k$ are *recomputed* from its ground state, Eq. (13); V^I_k (Eq. (15)) and $\lambda^{I,c}_k$ (Eq. (21)) follow from these. Note the consequences:

- the mixed Δ^{II} carried between iterations enters only through G in Eq. (40); the Δ^{II} used for V, λ^c and the energy is the one produced by $H_{\rm qp}$, so the equality of the quasiparticle and impurity density matrices, $\Delta^{II}_{\rm qp}=\Delta^{II}_{\rm new}$, holds only at self-consistency (this is the constraint of the λ^I-term of the Lagrangian, Section 3);
- nothing is solved iteratively inside an outer iteration: each step is a closed-form evaluation plus two diagonalizations of small matrices and two impurity solves, so there is no branch ambiguity to control.

What this replaces. An earlier version of the code instead *fitted* (λ^0, λ^1) at every iteration by a damped Newton solve (analytic Jacobian of the projector ρ^{qp}, trust-region step cap, line search) so that the qp density matrix reproduced a target Δ, the mixed impurity result. That is the reference code's *other* branch (`no_fit_scf F`, routine `OptLambd`); it is a different algorithm from the one the reference runs used here (`no_fit_scf T`), and on the reference seeds it diverged: the target Δ coming from the impurity is generally not reachable by the qp problem at fixed R, so the fit walked away from the physical branch (e.g. the physical filling drifted to $n_{\mathrm{phys}} \approx 1.02$ and then 1.28 within three iterations on the $U/t = 2$ seed). The fit machinery, the DIIS mixer, the "best iterate" safety net and the gauge-fixing routine were all removed together with this; the fit branch is not implemented.

7.2 Outer loop: mixing and convergence

After the impurity solves one has, per fragment, Δ^{II} and R^I (current) and $\Delta^{II}_{\mathrm{new}}$, R^I_{new} (Section 6.4). The reference `lin` updater then forms

$$X_{\mathrm{next}} \;=\; X + a_k \left(X_{\mathrm{new}} - X\right), \qquad X \in \{R^0, R^1, \Delta^{00}, \Delta^{11}\}, \qquad a_k = \begin{cases} 1 & k_{\mathrm{upd}} \le n_{\mathrm{eq}} \\ a_{\mathrm{mix}} & k_{\mathrm{upd}} > n_{\mathrm{eq}} \end{cases} \tag{41}$$

where $k_{\mathrm{upd}} = 1, 2, \ldots$ counts the calls of the mixer. That is, *plain substitution* during the first n_{eq} iterations (`gg_eq_time`, default 10), then linear mixing with weight a_{mix} (`gg_lin_mix`, default 0.65). The eigenvalues of each $\Delta^{II}_{\mathrm{next}}$ are then clipped to $[0, 1]$ (the reference `reg_mat` step; the clip is applied to the mixed matrix, not to the Δ that enters S).

Convergence test. With $E^{(k)}_{\mathrm{var}}$ the variational energy of iteration k (Eq. (39)), define

$$\delta E_k = E^{(k)}_{\mathrm{var}} - E^{(k-1)}_{\mathrm{var}} \quad (E^{(-1)}_{\mathrm{var}} \equiv 100), \tag{42}$$

$$\delta \Delta_k = \sum_I \left\| \Delta^{II}_{\mathrm{next},k-1} - \Delta^{II}_{\mathrm{next},k} \right\|_F \quad (\Delta_{\mathrm{next},-1} \equiv \text{the qp block at iteration 0}), \tag{43}$$

$$\delta R_k = \sum_I \left\| R^I_k - R^I_{\mathrm{next},k} \right\|, \tag{44}$$

the sum running over the two fragments. The loop stops when $|\delta E_k| < \mathrm{tol}_E$ *or* ($\delta \Delta_k < \mathrm{tol}_{\mathrm{mat}}$ *and* $\delta R_k < \mathrm{tol}_{\mathrm{mat}}$); the defaults are $\mathrm{tol}_E = \mathrm{tol}_{\mathrm{mat}} = 10^{-6}$, the reference values (`gg_abs_tol_E`, `gg_abs_tol_mat`). The maximum number of iterations defaults to 200. Nothing is returned other than the last iteration (there is no "best iterate" selection).

Reported state. At return, R^I is the *updated* R after the last mixing step (the C++ `final_R`), λ^I, $\lambda^{I,c}$, V^I, E_{var}, docc and n_{phys} are those of the last iteration.

Code: `run_gga(Ng, U, t=1.0, max_iter=200, tol_E=1e-6, tol_mat=1e-6, mix=0.65, eq_time=10, sqmat_shift=1e-9, verbose=False, Rg0=None, Rg1=None, lamg0=None, lamg1=None)`. `Rg1/lamg1` default to `Rg0/lamg0` when not given; if nothing is given at all, $R^0 = (0.9,\ 0.05,\ 0.05 \cdot 1.3,\ 0.05 \cdot 1.6, \ldots)$ (distinct ghost components, to break the exact ghost-exchange symmetry) and $\lambda^0 = 0$ (the C++ default is $R = (1, \ldots, 1)$, $\lambda = 0$).

7.3 The main self-consistency loop, in pseudocode

```
def run_gga(Ng, U, t=1.0, max_iter=200, tol_E=1e-6, tol_mat=1e-6,
           mix=0.65, eq_time=10, sqmat_shift=1e-9,
           Rg0=None, Rg1=None, lamg0=None, lamg1=None):
    Neff = 1 + Ng
    ops = build_impurity_ops(Neff, U, U / 2.0) # precomputed once
    R, lam = seeds(Rg0, Rg1, lamg0, lamg1) # lists [frag0, frag1]
    lamc = [0, 0]; V = [0, 0]; curr_E = 100.0; n_upd = 0

    for it in range(max_iter):
        if it == 0:
            Hqp, rho = qp_state(R, lam, t) # lam = input guess
            Delta = blocks(rho); prev_Delta = Delta
        else: # Eq. (lambda-update)
            lam = [-lamc[i] + grad_S(Delta[i], outer(V[i], R[i]), eps)
                   for i in (0, 1)]
            Hqp, rho = qp_state(R, lam, t)
            Delta = blocks(rho) # recomputed from Hqp

        V = [hybridization_V(rho, R, t, eps, i) for i in (0, 1)] # Eq. (V)
        lamc = [-lam[i] + grad_S(Delta[i], outer(V[i], R[i]), eps)
                for i in (0, 1)] # Eq. (lamc)

        imp = [impurity_solve(V[i], lamc[i], ops) for i in (0, 1)] # ED
        Delta_new = [I - imp[i].Delta_bb for i in (0, 1)] # Eq. (Delta)
        R_new = [S_inv(Delta_new[i]) @ imp[i].D for i in (0, 1)] # Eq. (R)

        E_var = 2*trace(rho @ (Hqp + blockdiag(lam))) + Eloc0 + Eloc1 # Eq. (Evar)

        n_upd += 1; a = 1.0 if n_upd <= eq_time else mix # Eq. (linmix)
        Delta_next = [Delta[i] + a*(Delta_new[i] - Delta[i]) for i in (0, 1)]
        R_next = [R[i] + a*(R_new[i] - R[i]) for i in (0, 1)]
        Delta_next = [clip_eigenvalues(D, 0, 1) for D in Delta_next]

        dD = sum(norm(prev_Delta[i] - Delta_next[i]) for i in (0, 1))
        dR = sum(norm(R[i] - R_next[i]) for i in (0, 1))
        dE = E_var - curr_E
        prev_Delta, R, Delta, curr_E = Delta_next, R_next, Delta_next, E_var
        if abs(dE) < tol_E or (dD < tol_mat and dR < tol_mat):
            break
    return R, lam, lamc, V, E_var, ... # last iteration
```

7.4 Validation against the reference C++ code, iteration by iteration

The reference code writes, for every iteration, the intermediate quantities (`curr_deltas`, `curr_Vs`, `curr_lambdcs`, `imp_1rdms`, `new_Rs`, E_{var}, $\delta\Delta$, δR) to `gg.out`. Two $U/t = 2$, $N_g = 2$ test cases, differing only in the seed (R, λ) read from the reference `guess_*.dat` files, were run with both codes: `backward` (a generic, asymmetric seed with Δ-eigenvalues $3.5 \times 10^{-5}, \frac{1}{2}, 0.99996$) and `forward` (the symmetric-solution seed with Δ-eigenvalues exactly $0, \frac{1}{2}, 1$, the one that produced NaN before the shift was introduced). The results agree at every step:

For `forward`, E_{var} goes -3.136429, -3.125710, -3.125044, -3.125003, -3.125000 (C++: -3.13643, -3.12571, and δR goes 5.892×10^{-3}, 3.665×10^{-4}, 2.290×10^{-5}, 1.431×10^{-6}, 8.950×10^{-8} (C++: the same

	$E_{\rm var}/t$		$\delta\Delta$		δR	
k	this code	C++	this code	C++	this code	C++
0	-3.113524	-3.11352	3.998×10^{-4}	3.998×10^{-4}	2.396×10^{-2}	2.396×10^{-2}
1	-3.124827	-3.12483	4.121×10^{-4}	4.121×10^{-4}	1.665×10^{-3}	1.665×10^{-3}
2	-3.125596	-3.12560	3.098×10^{-5}	3.098×10^{-5}	1.270×10^{-4}	1.270×10^{-4}
3	-3.125656	-3.12566	2.458×10^{-6}	2.458×10^{-6}	1.009×10^{-5}	1.009×10^{-5}
4	-3.125661	-3.12566	1.964×10^{-7}	1.964×10^{-7}	8.063×10^{-7}	8.063×10^{-7}

Table 1: Iteration-by-iteration comparison, `backward` case ($U/t = 2$, $N_g = 2$). The C++ $E_{\rm var}$ is printed with six significant figures in its log.

to the printed digits); in this case $\delta\Delta$ is at the rounding level ($\sim 10^{-9}$–10^{-10}) in both codes after the first iteration, so only its order of magnitude is meaningful. The final quasiparticle weight is $Z = 0.9375$ (= the eigenvalue of $R^\dagger R$ printed by the C++ code). In both cases the final R and λ written by `save_results` agree with the C++ `final_R_*.dat` and `final_lambda_*.dat` to $\lesssim 10^{-9}$ in every entry, and both runs converge in 5 iterations, as in the reference.

Cold starts. The reference code has no built-in initial guess: it reads R and λ from files. To test cold starts, the same seed files were given to both codes: $R = (1, 1, 1)$ ("ones"), the code's default $R = (0.9, 0.05, 0.065)$ ("pydef"), and a random $R \in [0.05, 1]^3$ ("rand1"), each with $\lambda = 0$, plus "onesN" and "pydefN" which add the same small symmetric random λ ($\sim 10^{-3}$) to break the exact degeneracy of $H_{\rm qp}$ at $\lambda = 0$; $U/t \in \{2, 4, 8, 10\}$, $N_g = 2$. Result: *all* 20 *runs converge with both codes, and the iteration counts and final Z are identical in all* 20 (energies agree to the digits the C++ log prints):

seed	$U/t = 2$	$U/t = 4$	$U/t = 8$	$U/t = 10$
ones	-3.1250 (8)	-4.5000 (12)	-8.0001 (189)	-10.0000 (35)
pydef	-3.1250 (7)	-4.5000 (11)	-8.0001 (187)	-10.0000 (34)
rand1	-3.1250 (7)	-4.5000 (11)	-8.0001 (188)	-10.0000 (34)
onesN	-3.1257 (7)	-4.5111 (15)	-8.1276 (33)	-10.1013 (16)
pydefN	-3.1257 (7)	-4.5111 (12)	-8.1276 (23)	-10.1013 (15)

Table 2: Final $E_{\rm var}/t$ (number of iterations) from cold seeds, $N_g = 2$; the C++ code gives the same numbers. Exact energies: -3.2361, -4.8284, -8.4721, -10.3852.

Reading Table 2. The fixed-point map has several solutions (as in any mean-field-like variational theory), and which one a cold start reaches depends on the seed, in the *same* way for both codes. With $\lambda = 0$ the seed sits on an exactly symmetric manifold and the run falls onto the plain-Gutzwiller branch: $Z = 0.75$, $E = -4.5$ at $U/t = 4$, and $Z \approx 5\times10^{-6}$, $E = -10$ at $U/t = 10$, i.e. the ghost orbitals stay unused ($N_g = 0$ gives the same values). With a tiny λ noise the run finds the ghost-assisted solution ($E = -4.5111$, -8.1276, -10.1013, the same as the seeded examples), which is lower in energy and closer to the exact values, so a cold start does *not* guarantee the best solution: at $\lambda = 0$ both codes converge, but to a higher-energy branch. Seeding `R.in`/`L.in` from a good solution, or a small random λ, is the reliable way to reach the lower branch.

Sensitivity, and why the FCIDUMP quantization is emulated. At $U/t = 10$ the reference code's $\lambda = 0$ run ends on the localized branch ($E = -10$), whereas the code without the FCIDUMP

quantization of Section 6.3, with seed perturbations of only 10^{-12}, lands on either branch depending on the perturbation ($E = -10.1013$ for some, -10.0000 for others), while the reference code gives $E = -10$ for all six perturbed seeds. With the quantization emulated the code gives $E = -10.0000$ in all six, in 34 iterations like the reference (and 188 iterations, $Z = 0.007935$ at $U/t = 8$, again as the reference).

Larger N_g: $N_g = 4$ gives the same energy as $N_g = 2$. The reference code finds the same E_{var} at $N_g = 4$ as at $N_g = 2$, and so does this code. Three seed types of Table 2 ("ones", "onesN", "pydefN", extended to $N_{\mathrm{eff}} = 5$ and given as identical seed files to both codes) were run at $N_g = 4$, $U/t \in \{2, 4, 8, 10\}$, i.e. 12 runs per code; all converge in both codes. With a small random λ the energies are -3.125661, -4.511137, -8.127658, -10.101314 (C++: -3.1257, -4.5111, -8.1277, -10.101), identical to the $N_g = 2$ values of Table 2; with $\lambda = 0$ they are -3.1250, -4.5000, -8.0001, -10.0000, again equal to $N_g = 2$ and to the C++. The two extra ghost orbitals therefore lower nothing for the dimer from these seeds, and the energies remain well above the exact ones (-3.2361, -4.8284, -8.4721, -10.3852); this does not rule out a lower $N_g = 4$ solution that these seeds do not reach. In 9 of the 12 runs the iteration count also equals the C++ one; in the other three (small-λ seeds at $U/t = 8$ and 10) the final energy is the same but the count differs (30 against 37, 12 against 16, and 32 against 29); this was not investigated further.

8 Observables reported by the code

- **Quasiparticle weight** $Z^I = \|R^I\|^2 = (R^I)^T R^I$, reported per fragment as `Z0`, `Z1`.
- **Variational energy** E_{var}, Eq. (39).
- **Double occupancy** $\mathrm{docc} = \frac{1}{2}(\mathrm{docc}^0 + \mathrm{docc}^1)$, the fragment-averaged impurity double occupancy (Section 6.3), directly comparable to `exact_dimer.py`'s `docc/site` column.
- **Physical filling** $n_{\mathrm{phys}} = \frac{1}{2}(n^0_{\mathrm{phys}} + n^1_{\mathrm{phys}})$.
- **Final parameters** $R^0, R^1, \lambda^0, \lambda^1, \lambda^{0,c}, \lambda^{1,c}, V^0, V^1$ of the last iteration (R after the last mixing step, as in the reference), all returned in the result dictionary (together with a `converged` flag) and, from the command line, written to `<prefix>_R.dat`, `<prefix>_lambda.dat`, `<prefix>_lambda_c.d` (one row-block per fragment: atom 0 then atom 1).
- **Spectral function** $A(\omega)$ (Section 9), computed only if `--omega`, `--eta`, `--domega` are given, written to `A_omega.txt`.

9 Post-processing: quasiparticle Green's function and spectral function

Once the outer loop of Section 7 has converged, the same converged $H_{\mathrm{qp}}(R^0, R^1, \lambda^0, \lambda^1)$ (Eq. (11)) can optionally be reused, as a cheap post-processing step, to evaluate the one-body quasiparticle Green's function and, from it, the physical spectral function of the dimer. This is a diagnostic/-plotting step only: it does not feed back into the self-consistency loop.

Quasiparticle Green's function. On a uniform real-frequency grid running from $-\omega_{\max}$ to $+\omega_{\max}$ in steps of $d\omega$ (all three, $\omega_{\max}$, the broadening η, and $d\omega$, are supplied by the user – Section 10), the $2N_{\text{eff}} \times 2N_{\text{eff}}$ retarded quasiparticle Green's function is

$$G(\omega) \;=\; \big[(\omega + i\eta)\,\mathbf{I} - H_{\text{qp}}\big]^{-1}, \tag{45}$$

with H_{qp} the same fixed, converged matrix of Eq. (11) (the same one for both spin channels, since H_{qp} does not carry a spin index) and $\mathbf{I}$ here the $2N_{\text{eff}} \times 2N_{\text{eff}}$ identity.

Transformation to the physical basis. The physical creation operator of fragment I is related to the qp/bath operators by $c_I^\dagger \to \sum_a R_a^I d_{Ia}^\dagger$ (Eq. 14 of `NotesOngGut.pdf`, already used in Section 3). Collecting both fragments' R vectors into a single block-diagonal $2N_{\text{eff}} \times 2$ matrix,

$$R_{\text{mat}} \;=\; \begin{pmatrix} R^0 & 0 \\ 0 & R^1 \end{pmatrix} \in \mathbb{R}^{2N_{\text{eff}} \times 2}, \tag{46}$$

the 2×2 physical-basis (site-resolved) Green's function is the congruence transform

$$G_{\text{phys}}(\omega) \;=\; R_{\text{mat}}^T\, G(\omega)\, R_{\text{mat}}, \tag{47}$$

whose diagonal entries are the site-0/site-1 physical Green's functions and whose off-diagonal entries are the inter-site physical Green's function mediated by the qp sector.

Spectral function. Tracing Eq. (47) sums the two sites' contributions; the total physical spectral function is

$$\boxed{A(\omega) \;=\; -\frac{2}{\pi}\,\mathrm{Im}\,\mathrm{Tr}\big[G_{\text{phys}}(\omega)\big]} \tag{48}$$

The factor of 2 accounts for both spin channels: H_{qp}, and hence $G(\omega)$ and $G_{\text{phys}}(\omega)$, is identical for $\uparrow$ and $\downarrow$ (the paramagnetic solutions this code targets never distinguish them), so the total spectral weight is twice the single-spin trace computed from Eq. (45)–(47).

Sum rule (validation). For a normalized single-particle Green's function, $\int_{-\infty}^{\infty} A(\omega)\, d\omega$ must equal the total number of physical (site, spin) degrees of freedom, i.e. 2 sites $\times$ 2 spins = 4. Numerically integrating $A(\omega)$ from `examples/Ng2_U10` ($U/t = 10$, $N_g = 2$) over $\omega \in [-15, 15]$ with $\eta = 0.05$, $d\omega = 0.05$ gives $\int A\, d\omega \approx 3.96$, within the expected error from the finite frequency cutoff and the Lorentzian broadening tails that extend (slightly) past $\pm\omega_{\max}$ – confirming both the overall normalization and the factor of 2 in Eq. (48).

Code: `compute_spectral_function(R0, R1, lam0, lam1, t, omega_max, eta, domega)` builds H_{qp} and R_{mat} once, then loops over the frequency grid (built with `numpy.arange` padded by half a step so the closed endpoint $+\omega_{\max}$ survives floating-point round-off) evaluating Eqs. (45)–(48) at each point; `save_spectral_function` writes the two-column result (`omega`, `A(omega)`) to `A_omega.txt`.

10 Command-line interface and input file formats

```
python3 gga_dimer.py --U <U/t> --Ng <even integer>
                     [--t <hopping, default 1.0>]
                     [--max-iter <default 200>]
                     [--sqmat-shift <default 1e-9>]
                     [--no-fcidump-rounding]
                     [--mix <default 0.65>] [--eq-time <default 10>]
                     [--tol-E <default 1e-6>] [--tol-mat <default 1e-6>]
                     [--quiet]
                     [--R-file <path, default R.in>]
                     [--lam-file <path, default L.in>]
                     [--omega <omega_max> --eta <eta> --domega <step>]
```

The last five numerical options correspond one to one to the reference input keys `gg_sqmat_shift`, `gg_lin_mix`, `gg_eq_time`, `gg_abs_tol_E` and `gg_abs_tol_mat` (and `--max-iter` to `gg_maxits`); `--no-fcidump-round` switches off the emulation of the reference FCIDUMP hand-over of Section 6.3. The option `--regularize` of the earlier version no longer exists (the clips and caps it switched on are not part of the reference algorithm). Called with no `--U`/`--Ng`, the script instead prints a default sweep over $N_g \in \{0, 2, 4\}$ and $U/t \in \{0, 1, 2, 4, 8\}$, using the default cold-start seed for every point (no external guess files are read in this mode).

`--omega, --eta, --domega.` Optional, and only meaningful together with `--U`/`--Ng`; the three must be given together (any one given without the other two is a CLI error). When given, after the self-consistency loop converges the code evaluates the spectral function of Section 9 on the grid $-$`omega` to $+$`omega` in steps of `domega`, with broadening `eta`, and writes `A_omega.txt` in the current directory. None of the three has a default: the user must supply all three explicitly, since there is no physically motivated default frequency window, broadening, or resolution – these depend entirely on the energy scales (U, t, the converged λ, λ^c) of the particular run.

`R.in.` Either *one* row of N_{eff} numbers (used as a symmetric guess, $R^0 = R^1 =$ that row) or *two* rows of N_{eff} numbers each (fragment 0's row, then fragment 1's row), free-format whitespace- or comma-separated.

`L.in.` Either N_{eff} rows of N_{eff} numbers (one symmetric matrix, used for both fragments) or $2N_{\text{eff}}$ rows of N_{eff} numbers (fragment 0's $N_{\text{eff}} \times N_{\text{eff}}$ block, stacked on top of fragment 1's block), free-format. To seed a run from the reference C++ code's `guess_L1.dat`, `guess_L2.dat` (one $N_{\text{eff}} \times N_{\text{eff}}$ matrix per fragment) and `guess_R1.dat`, `guess_R2.dat` (one row per fragment), simply concatenate them: `L.in` = `guess_L1.dat` then `guess_L2.dat`, and likewise for `R.in`.

Both files are read automatically if present in the working directory (or at the path given by `--R-file`/`--lam-file`); if absent, the default cold-start seed of Section 7.2 is used. Every run writes its final (R, λ, λ^c) back out in exactly this stacked, two-row-block format (`save_results`), so a converged run's output can be fed back in directly as the next run's guess.

11 Worked examples and current validation status

Two examples, seeded from an independent reference implementation's own iterates, are kept under `examples/`. In addition, the two reference test cases of Section 7.4 (`forward` and `backward`, with the reference code's complete `gg.out` logs) are the primary, iteration-by-iteration validation of everything in this document.

Example	N_g	U/t	seed type	this code's $E_{\rm var}/t$	reference $E_{\rm var}/t$
`examples/Ng2_U10`	2	10	symmetric ($R^0{=}R^1$)	-10.101314	-10.101316
`examples/Ng2_U2`	2	2	asymmetric ($R^0{\neq}R^1$)	-3.125661	-3.12566
`forward` (Sec. 7.4)	2	2	symmetric-solution seed	-3.125000	-3.125

Table 3: Seeded reference examples and the agreement obtained with the algorithm of Section 7 (the `backward` case is the same physical solution as `examples/Ng2_U2`, Table 1).

12 Are the converged points solutions? Stationarity and second-order tests

Convergence of the loop of Section 7 means that $E_{\rm var}$, R and (the changes of) $\Delta_{\rm next}$ stop changing. It does *not* by itself show that the converged point is a stationary point of the Lagrangian of Section 3, nor that it is a minimum. Two diagnostic scripts, `check_stationarity.py` and `check_hessian.py`, test both for the $U/t = 2$, $N_g = 2$ solutions. They only *read* the solver (`gga_dimer.py` is not modified).

Lagrangian used. In the conventions of `gga_dimer.py` (spin summed, $S(\Delta) = \sqrt{\Delta(\mathbf{I}-\Delta)+\varepsilon}$),

$$\begin{aligned} L = & -2t\,(R^0\rho^{01}R^1 + R^1\rho^{10}R^0) + \sum_I \Big\{ \langle\Phi^I|H_{\rm loc}|\Phi^I\rangle \\ & + 4V^I\cdot(D^I - S(\Delta^I)R^I) \\ & + 2{\rm Tr}\big[\lambda^{I,c}(\Delta^I + \Delta^{I,{\rm bb}} - \langle\Phi^I|\Phi^I\rangle)\big] \\ & + 2{\rm Tr}\big[\lambda^I(\Delta^I - \rho^{II})\big]\Big\}, \end{aligned} \tag{49}$$

with $\rho = CC^T$ the 1-RDM of the qp Slater determinant (C: the $N_{\rm eff}$ occupied orbitals of $2N_{\rm eff}$) and Φ^I the impurity state in the $N_\uparrow = N_\downarrow$ sector. Its stationarity conditions are exactly the equations of the code: $\partial L/\partial V = 4(D - SR)$ (R update), $\partial L/\partial\lambda^c = 2(\Delta + \Delta^{\rm bb} - 1)$ (bath density $= 1-\Delta$), $\partial L/\partial\lambda = 2(\Delta - \rho^{II})$, $\partial L/\partial R^0 = -4t\rho^{01}R^1 - 4S(\Delta^0)V^0$ (Eq. (15)), $\partial L/\partial\Delta = 2(\lambda+\lambda^c) - 2G(\Delta, VR)$ (Eq. (20)), $H_{\rm imp}\Phi = E\Phi$, and $[H_{\rm qp}, \rho] = 0$. Each block was evaluated analytically and by central finite differences of L itself, at the converged point and at a point with R^0_0 shifted by 10^{-2} (which must be, and is, non-stationary: gradients $\sim 10^{-2}$).

	backward	forward	third
$E_{\rm var}/t$	-3.125661	-3.125000	-3.125324
Z	0.9999	0.9375	0.9683
$\max\lvert\partial L/\partial x\rvert$, $x = R, V, \Delta, \rho, \Phi$	$\lesssim 10^{-9}$	$\lesssim 10^{-8}$	$\lesssim 10^{-7}$
$\max\lvert\partial L/\partial\lambda^c\rvert$	$\lesssim 10^{-9}$	**1.94**	**1.95**
L at the point	$E_{\rm var}$	-7.15	-7.07

Table 4: Stationarity of L at three converged solutions. "backward" and "forward" are the two solutions reached from the reference seeds of Section 7.4; "third" is reached from the forward one by a relative perturbation of 10^{-3} of R, λ. The finite-difference Δ block is limited to $\sim 10^{-5}$–10^{-7} by the stiffness of S (its analytic value is $\lesssim 10^{-7}$).

Results ($U/t = 2$, $N_g = 2$).

- **Backward** is a stationary point of L in every block.
- **Forward** ($Z = 0.9375$, the plain Gutzwiller value $1 - (U/8)^2$) is *not*: $\partial L/\partial\lambda^c = 2$ exactly in the two directions where Δ has eigenvalue $n = 0$ and $n = 1$, and $L \neq E_{\mathrm{var}}$. In those directions $R = V = 0$ (the ghost orbitals are inert), $G(\Delta, VR) = 0$, so $\lambda^c = -\lambda$, and the impurity ghost has $\Delta^{\mathrm{bb}} = n$ instead of $1 - n$: the constraint is violated by ~ 1 there. On the active direction ($n = \frac{1}{2}$) all conditions hold to 10^{-16}. A plain $N_g = 0$ Gutzwiller run (same E and Z) is stationary in every block. The recorded reference log of this case also has $\lambda^c = -\lambda$.
- **Third** ($E = -3.125324$, one inert ghost per fragment) is again not stationary in λ^c. Moreover $\mathrm{Tr}\,\Delta_{\mathrm{qp}} = (2.50, 0.50)$ while the impurity gives $\mathrm{Tr}(\mathbf{I} - \Delta^{\mathrm{bb}}) = (1.50, 1.50)$: the particle numbers are inconsistent by one electron, so no feasible point with this R exists in the $N_\uparrow = N_\downarrow$ sector.

The reason such points pass the convergence test is that it monitors the change of Δ_{next} and R, not the difference $\Delta_{\mathrm{qp}} - \Delta_{\mathrm{new}}$; the mixed Δ enters only through G (Section 7.1), so a constant mismatch is invisible. They are fixed points of the iteration, but not stationary points of L.

Second-order test. At a stationary point that satisfies the constraints the solution is a strict local minimum of the constrained energy if the Hessian of the Lagrangian, at fixed multipliers, is positive definite on the tangent space of the constraints ($D - SR = 0$, $\Delta + \Delta^{\mathrm{bb}} - \langle\Phi|\Phi\rangle = 0$, $\Delta - \rho^{II} = 0$, $\langle\Phi|\Phi\rangle = 1$). The variables are R^I, Δ^I, the occupied-to-virtual rotations κ of Ψ (99 variables, 32 constraints for $N_g = 2$), all blocks analytic, including the exact second derivative of $S(\Delta)$ from the Sylvester equation. It was validated against finite differences (Hessian blocks to 3×10^{-7} for entries up to 29; Jacobian columns to 3×10^{-9}).

- **Backward is a strict local minimum**: the reduced Hessian has 61 positive, 0 negative and 6 zero eigenvalues; the 6 zero modes are exactly the $O(N_{\mathrm{eff}})$ gauge rotations of the ghost basis of the two fragments ($3 + 3$). The smallest non-gauge eigenvalue is 1.78. Along an actual feasible curve $x(s)$ (Gauss–Newton projection onto the constraints) the softest mode gives $\Delta E = +2.4 \times 10^{-5}$ and $+2.1 \times 10^{-5}$ for $\pm s$ at $s = 0.005$, against the predicted $+2.2 \times 10^{-5}$, and follows s^2 up to $s = 0.05$.
- **Forward**: as returned by the solver it violates the constraints (violation 0.97), so a constrained Hessian is not defined there. Making it feasible by hand (flipping the ghost bath occupations, an *excited* eigenstate of the same H_{imp}, same E_{loc}) makes L stationary ($\lesssim 10^{-8}$), but that is a different point from the solver's. At it the reduced Hessian has 18 negative, 6 zero and 47 positive eigenvalues, yet the constraint qualification fails (Jacobian rank 28 of 32) and along the negative directions the energy on actual feasible curves changes by only $\sim -4 \times 10^{-12}$ (predicted -5×10^{-5} to -5×10^{-3}): the negative curvature is spurious and the test is *inconclusive*. What is established is that forward has a higher energy than backward and that it is not a stationary point of L.
- **Stability of the iteration.** Perturbing the converged forward R, λ by a relative size δ and rerunning the solver: for $\delta \leq 10^{-6}$ it returns to forward; for $\delta \geq 10^{-4}$ it goes to backward (-3.125661) or to the third solution (-3.125324), with or without the FCIDUMP rounding. Forward is a weak attractor of the iteration with a very small basin; this is a statement about the iteration, not about energetic descent.

NaN of $\sqrt{\Delta(\mathbf{I}-\Delta)}$. With the shift, $S(\Delta)$ is real only if every eigenvalue satisfies $n(1-n)+\varepsilon \geq 0$, i.e. $n \geq -\varepsilon$ and $n \leq 1+\varepsilon$ (to first order); otherwise the square root is `NaN`. The forward and third solutions have eigenvalues at 0 and 1, so any step larger than $\varepsilon = 10^{-9}$ leaves the domain. This produced NaN in the diagnostics whenever a finite-difference step (10^{-6}, 10^{-8}) or a feasible curve pushed such an eigenvalue past the boundary (for example the Jacobian by finite differences with step 10^{-8}, or one side of a curve at $s = 0.05$ for backward); the diagnostics use a Δ step of $10^{-10} < \varepsilon$ and guard against non-finite values. The solver itself produced no NaN in any of the runs reported here (the shift covers rounding of the eigenvalues of Δ to -10^{-9}, and the mixed Δ is clipped to $[0,1]$), but noise of the eigenvalues of Δ larger than ε would produce it; the unshifted formula gave NaN on the forward seed (Section 6.1).

13 Generalization: a site-splitting potential ΔV

`gga_dimer.py` (and `not_imp_gga_dimer.py`, Section "Generalization: a site-splitting potential ΔV" of `doc/not_impurity.tex`) optionally generalizes the physical Hamiltonian Eq. (1) to

$$H_{\text{phys}} \;\to\; H_{\text{phys}} \;+\; \frac{\Delta V}{2}\,(n_0 - n_1), \qquad n_i = n_{i\uparrow} + n_{i\downarrow}, \tag{50}$$

site 0 raised and site 1 lowered by $\Delta V/2$ ($\Delta V = 0$ recovers Eq. (1) exactly); this is the same model generalization as `exact_dimer_deltaV.py`/`scan_ed_deltaV/ED_deltaV.tex`, here combined with the gGA approximation instead of exact diagonalization. CLI flag: `--deltaV` (default 0.0) on both scripts.

Only one new term in the Lagrangian. The new term of Eq. (50) is a bare *one-body*, fragment-*diagonal* operator ($\epsilon_0 = +\Delta V/2$, $\epsilon_1 = -\Delta V/2$; no $I \neq J$ part), so it enters the Lagrangian Eq. (5) exactly like the kinetic term, but with $I = J$ instead of $I \neq J$:

$$L \;\to\; L \;+\; \sum_I \epsilon_I\,(R^I)^T \langle \Psi_{\text{qp}} | d_I^\dagger d_I | \Psi_{\text{qp}} \rangle\, R^I. \tag{51}$$

Only two of the Euler-Lagrange equations of Section 3.1 depend on $|\Psi_{\text{qp}}\rangle$ or R^I, so only two equations of this document change – H_{qp} (Eq. (11), from varying $\langle\Psi_{\text{qp}}|$) and the V-fit (Eq. (15), from varying R^I, Section 3.1); H_{loc} (Eq. (25)), $\lambda^{I,c}$ (Eq. (21)) and the variational energy's *formula* $E_{\text{var}} = E_{\text{qp}} + E_{\text{loc}}$ are all unchanged – the new physics enters only through H_{qp} and V.

$$\boxed{H_{\text{qp}} \;=\; \begin{pmatrix} -\lambda^0 + \frac{\Delta V}{2} R^0 (R^0)^T & -t\, R^0 (R^1)^T \\ -t\, R^1 (R^0)^T & -\lambda^1 - \frac{\Delta V}{2} R^1 (R^1)^T \end{pmatrix}} \tag{52}$$

(reduces to Eq. (11) at $\Delta V = 0$). Because $H_{\text{qp}} + \text{diag}(\lambda^0, \lambda^1)$ keeps exactly the ΔV term and loses the λ term, $E_{\text{qp}} = 2\,\text{Tr}[\rho^{\text{qp}}(H_{\text{qp}} + \text{diag}(\lambda^0, \lambda^1))]$ (Section 6's variational-energy formula) automatically picks up the correct $2\epsilon_I (R^I)^T \Delta^{II} R^I$ contribution with no change to that formula itself.

$$\boxed{\sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})}\, V^I \;=\; \epsilon_I\, \Delta^{II} R^I \;-\; t\, \Delta^{IJ} R^J, \qquad J \neq I,} \tag{53}$$

i.e. V^I gains a same-fragment term with the *same* per-term coefficient convention as the already-validated neighbor term of Eq. (15) (the general, multi-orbital RISB form being $X_i = \sum_j (h_0)_{ij}\, \rho_{ij} R^j$, summed over *all* j including $j = i$, with h_0 the bare one-body Hamiltonian: hopping off-diagonal, ϵ_I on the diagonal).

Code: `qp_state(R, lam, t, deltaV)` builds Eq. (52); `hybridization_V(rho, R, t, shift, i, deltaV)` builds Eq. (53).

A bug found and fixed while adding this. An earlier version of this generalization added the ΔV term to $H_{\rm qp}$ (Eq. (52)) but *not* to the V-fit, reasoning (wrongly) that V^I is fixed only by the V^I-term of the Lagrangian, which Eq. (51) does not touch – but the V-fit comes from varying R^I (Section 3.1, "This – not the V^I-term's own stationarity – is where the V-fit equation comes from"), and R^I *does* appear in Eq. (51). This was caught by `not_imp_gga_dimer.py`'s independent KKT check, "$H_{\rm imp}\phi = E\phi$" (its ΔV section in `doc/not_impurity.tex`): with the old (uncorrected) V-fit this residual was ~ 0.3 at $\Delta V \neq 0$ (vs. the $\lesssim 10^{-6}$ baseline at $\Delta V = 0$) even though `gga_dimer.py`'s own internal self-consistency check ($\Delta^{\rm qp}$ vs. $\Delta^{\rm new}$, Section 7) was satisfied to $\sim 10^{-13}$ – i.e. the fixed-point iteration was internally consistent but converging to the *wrong* fixed point, not a numerical non-convergence. A sweep of candidate coefficients for the new V-fit term (comparing against the KKT residual) confirmed the coefficient of Eq. (53); after the fix the residual is $\sim 10^{-7}$–10^{-8}, matching the $\Delta V = 0$ baseline, and `gga_dimer.py` and `not_imp_gga_dimer.py` agree on $E_{\rm var}$, Z^0, Z^1 to $\sim 10^{-8}$–10^{-10} (checked for $N_g = 0$, several $(U, \Delta V)$; see its ΔV section in `doc/not_impurity.tex` for the $N_g = 2$ caveat).

Validation. (1) *Exact $U = 0$ limit*: at $U = 0$ the gGA ansatz is exact, and `run_gga` converges to $R^0 = R^1 = \mathbf{1}$ (no renormalization, $Z^0 = Z^1 = 1$, any N_g) with $H_{\rm qp}$'s eigenvalues matching $\pm\xi = \pm\sqrt{t^2 + (\Delta V/2)^2}$ – the exact one-body (free-fermion) problem with ΔV – to 7+ digits, for several ΔV. (2) *Finite-difference gradient check* of the direct-minimization energy functional with $\Delta V \neq 0$ (`not_imp_gga_dimer.check_gradients`, see `doc/not_impurity.tex`): $\lesssim 10^{-10}$. (3) *Cross-validation* between the two independent formulations, described above. No comparison against `exact_dimer_deltaV.py` (exact diagonalization) is made here: gGA is a variational approximation, exact only at $U = 0$ or in special limits, so agreement with ED is not expected at generic $(U, \Delta V)$ (see `plot_eqp_3d.py` for how the $\Delta V = 0$ case is compared to ED only qualitatively, via the spectral function).

Known caveat: $N_g = 2$ convergence. For $N_g \geq 2$, `run_gga`'s fixed-point iteration can converge to a spurious, highly symmetric "sleeping ghost" fixed point ($Z^0 = Z^1 \approx 0$) regardless of the initial guess, including when seeded *at* the true, lower-energy solution found by `not_imp_gga_dimer.py` – i.e. the true solution is not even a stable fixed point of the iteration's local basin structure for the $(N_g, U, \Delta V)$ combinations tried. This is the same "sleeping ghost"/multiple-fixed-point phenomenon already documented for $\Delta V = 0$ (the reason `scan_U_forward.py`/`scan_U_backward.py` exist as two different starting-guess branches); ΔV appears to make it worse for at least some parameter combinations tested, since the default initial guess ($R^0_0 = 0.9$, small symmetric-looking tail) was never tuned for a site-asymmetric problem. `not_imp_gga_dimer.py`'s direct minimization does not have this failure mode (confirmed by `hessian_analysis`'s KKT stationarity, independent of `analyse`'s λ-recovery machinery) and is the more reliable solver for $N_g \geq 2$, $\Delta V \neq 0$ until `run_gga`'s guess-generation is revisited.

14 References

1. N. Lanatà *et al.*, "Phase Diagram and Electronic Structure of Praseodymium and Plutonium," Phys. Rev. X **5**, 011008 (2015); and N. Lanatà *et al.*, Phys. Rev. X **7**, 031013 (2017) – original ghost Gutzwiller approximation.

2. F. Mejuto-Zaera and M. Fabrizio, "Efficient computational screening of strongly correlated materials: Multi-orbital phenomenology within the ghost Gutzwiller approximation," arXiv:2305.03329 – procedural formulation for periodic lattices (source of the general $H_{\rm qp}$, filling, and update-equation structure specialized in Section 5).

3. F. Mejuto-Zaera, "Quantum embedding for molecules using auxiliary particles – the ghost Gutzwiller ansatz," Faraday Discuss. **254**, 653 (2024), arXiv:2403.05157 – finite/molecular (non-periodic) formulation; the direct source of Eqs. (15)–(39) (Eqs. 7–11 of that paper) as specialized here to the two-site, two-independent-fragment dimer.

4. C. Mejuto-Zaera, "Notes on (ghost) Gutzwiller" (unpublished notes, `NotesOngGut.pdf`) – the source of the Lagrangian derivation underlying Eq. (20) (its Eq. 29) and of the impurity Hamiltonian's exact operator ordering, Eq. (25) (its Eq. 30), including the sign subtlety of Eq. (28).

5. Independent reference implementation (C++, `gGut` package): used throughout as ground truth: the whole self-consistency cycle of Sections 4–7 is a step-by-step port of `GhostGutzwiller::Run` (`ggutzwiller.c++`), with the helper routines `GetMatSqrt` (`utils.c++`), `eigspaceOneRDMDerivator` (`delta_derivative.c++`), `RSImpMod::GetGSlocX` / `GetQpGSloc1RDM_full` (`rsimp_mod.c++`), `LinearMixer` (`scf_updater.c++`) and the termination tracker (`ggtermtrack.c++`); validated against its `gg.out` iteration logs (Section 7.4) and its `2.0.out`, `10.0.out` logs.